

Demo: Materialized View

Summary

Created a Materialized View named `tickets_mv` in Amazon Redshift using sales and event data, validated cached query results, updated underlying table data, refreshed the Materialized View manually, and confirmed that refreshed results reflected the latest sales values successfully.

What is a Materialized View?

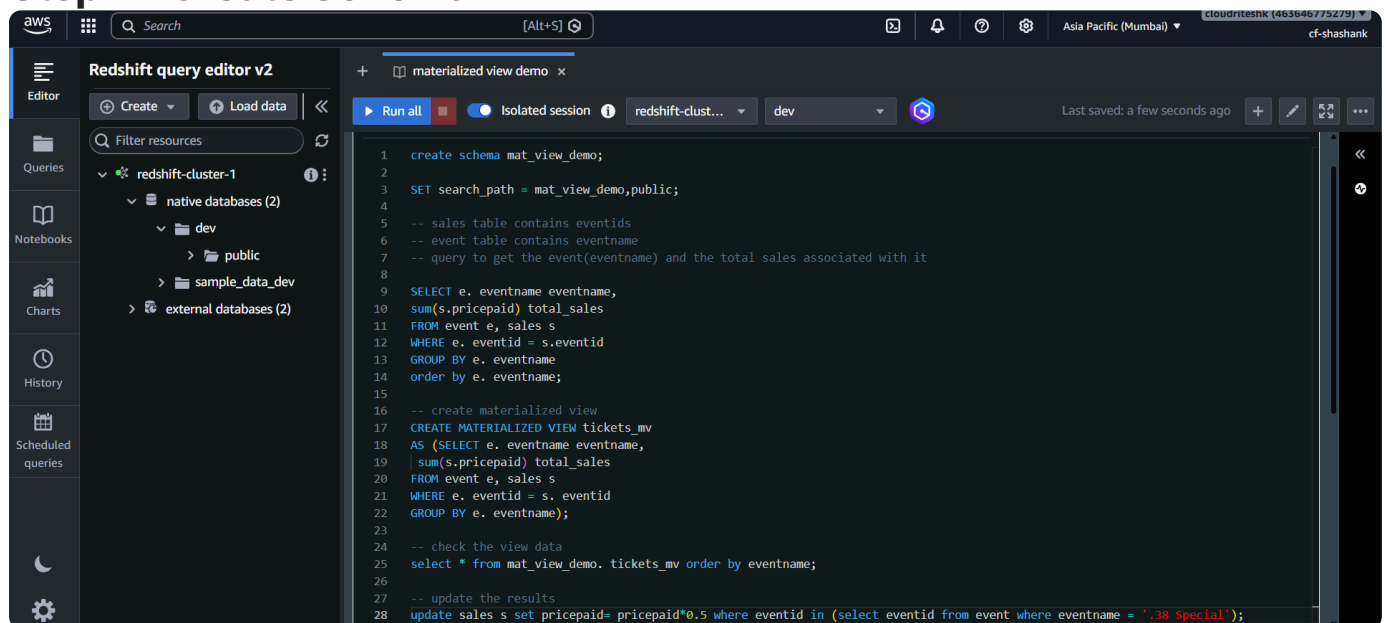
A Materialized View:

- Stores query results physically
- Helps improve performance for repetitive queries
- Avoids executing complex joins repeatedly

Useful for:

- Dashboards
- Reports
- Frequently accessed analytics queries

Step 1 – Create Schema



Created a separate schema for demo data:

```
CREATE SCHEMA mat_view_demo;
```

Purpose:

- Keep materialized view objects separate from other database objects.

Step 2 – Set Search Path

Used:

SET search_path

Purpose:

- Avoid writing schema prefixes repeatedly in queries.

Step 3 – Run Base Query

The screenshot shows the AWS Redshift Query Editor v2 interface. The left sidebar contains navigation options: Editor, Queries, Notebooks, Charts, History, and Scheduled queries. The main editor area displays a SQL query with line numbers 8 through 28. The query includes a SELECT statement with a JOIN, a CREATE MATERIALIZED VIEW statement, and an UPDATE statement. Below the query editor, the 'Result 1 (100)' tab is active, showing a table with two columns: 'eventname' and 'total_sales'. The table contains two rows of data.

```
8 SELECT e.eventname eventname,
9 sum(s.pricepaid) total_sales
10 FROM event e, sales s
11 WHERE e.eventid = s.eventid
12 GROUP BY e.eventname
13 ORDER BY e.eventname;
14
15
16 -- create materialized view
17 CREATE MATERIALIZED VIEW tickets_mv
18 AS (SELECT e.eventname eventname,
19 sum(s.pricepaid) total_sales
20 FROM event e, sales s
21 WHERE e.eventid = s.eventid
22 GROUP BY e.eventname);
23
24 -- check the view data
25 select * from mat_view_demo.tickets_mv order by eventname;
26
27 -- update the results
28 update sales s set pricepaid= pricepaid*0.5 where eventid in (select eventid from event where eventname = '.38 Special');
```

eventname	total_sales
.38 Special	134386
3 Doors Down	125816

The query:

- Joined:
 - sales
 - event
- Returned:
 - Event name
 - Associated total sales

Why JOIN?

- sales table contains:
 - eventid
 - sales values
- event table contains:
 - event names

The query returned: 100 rows

Why Use Materialized View?

This query may:

- Return large datasets
- Be executed multiple times daily
- Be used by dashboards and reports

Instead of running the query every time:

- Results can be cached using a Materialized View.

This improves:

- Query performance
- Dashboard response time

Step 4 – Create Materialized View

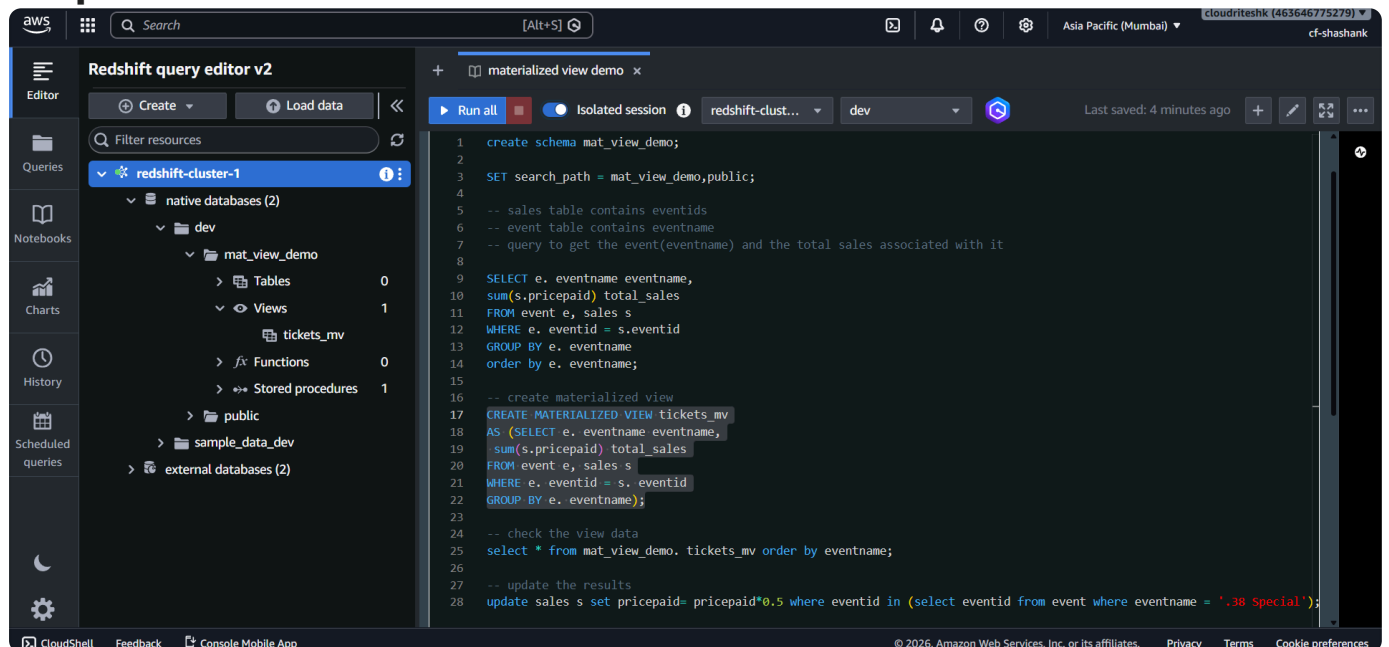
Used:

CREATE MATERIALIZED VIEW

Materialized view name: `tickets_mv`

The previously executed query was used inside the view definition.

Step 5 – Validate Materialized View



After refreshing the left panel:

Inside schema: `mat_view_demo`

Observed:

- 1 View
- 1 Internal Table

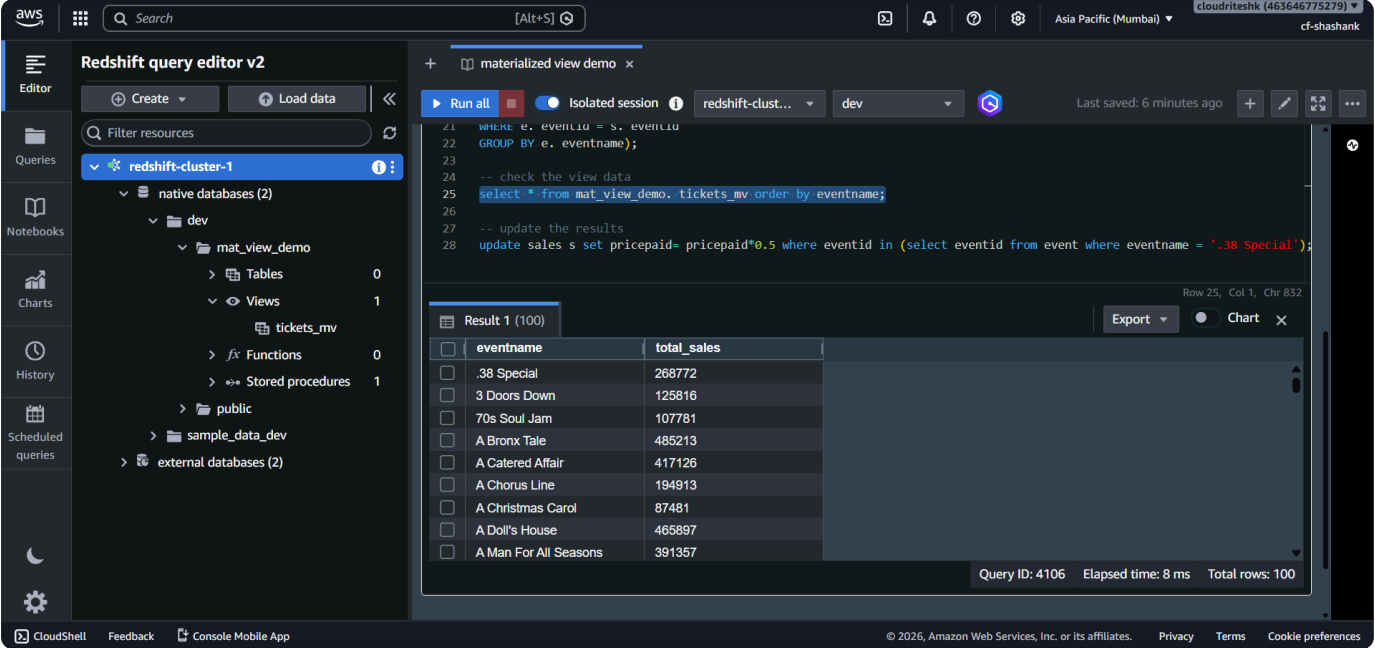
Important Note

The internal table:

- Is managed automatically by Redshift
- Stores cached data internally
- Is not directly used by users

We interact only with: `tickets_mv`

Step 6 – Query the Materialized View



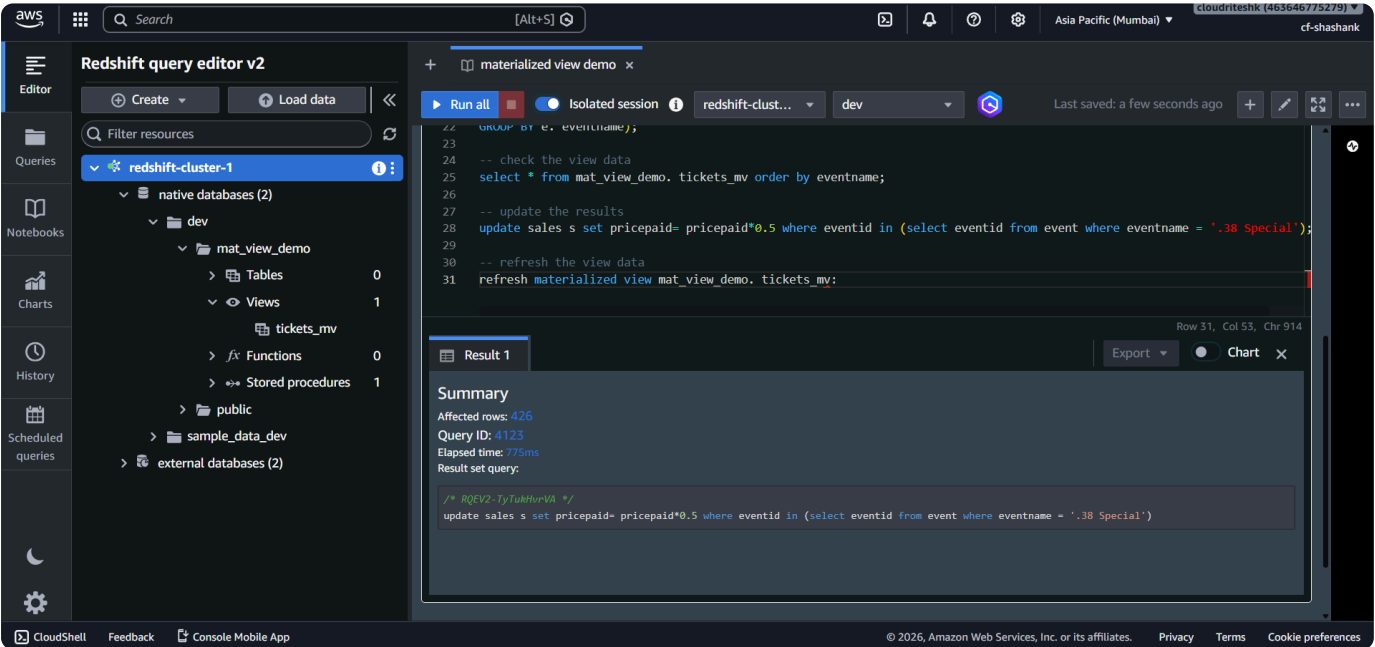
Executed query on: tickets_mv

Result: 100 rows

Same output as the original query was returned successfully.

Step 7 – Update Base Table Data

Updated the sales table.



Example Used

Event: .38 Special

Action:

- Doubled the sales amount for all related rows.

Expected sales:

- Increased from: `~0.26 million`

to: `~0.5 million`

Updated rows: `426 rows`

Step 8 – Check Materialized View Data

Queried: `tickets_mv`

The screenshot shows the AWS Redshift Query Editor v2 interface. The left sidebar displays the 'Queries' section with a tree view showing the database structure: 'native databases (2)' > 'dev' > 'mat_view_demo' > 'Views' > 'tickets_mv'. The main editor area shows a SQL query with line numbers 23 to 31. The query includes a 'select' statement, an 'update' statement, and a 'refresh materialized view' statement. Below the query editor, the 'Result 1 (100)' table is displayed, showing the data for the 'tickets_mv' view. The table has two columns: 'eventname' and 'total_sales'. The data is as follows:

eventname	total_sales
.38 Special	268772
3 Doors Down	125816
70s Soul Jam	107781
A Bronx Tale	485213
A Catered Affair	417126
A Chorus Line	194913
A Christmas Carol	87481
A Doll's House	465897
A Man For All Seasons	391357

The bottom of the interface shows the query ID (4164), elapsed time (6 ms), and total rows (100).

Observation:

- Old values were still returned.

Why?

Materialized Views use cached data.

They do not automatically refresh after table updates unless refreshed explicitly.

Step 9 – Refresh Materialized View

The screenshot shows the AWS Redshift Query Editor v2 interface after refreshing the materialized view. The left sidebar is the same as in Step 8. The main editor area shows the same SQL query, but the 'refresh materialized view' statement is now highlighted. Below the query editor, the 'Result 1' section shows a 'Summary' box with an 'Info' message: 'Materialized view tickets_mv was incrementally updated successfully.' Below the summary box, it shows 'Returned rows: 0', 'Query ID: 4193', 'Elapsed time: 1.6s', and 'Result set query:'. The bottom of the interface shows the query ID (4193), elapsed time (1.6s), and result set query.

Used:

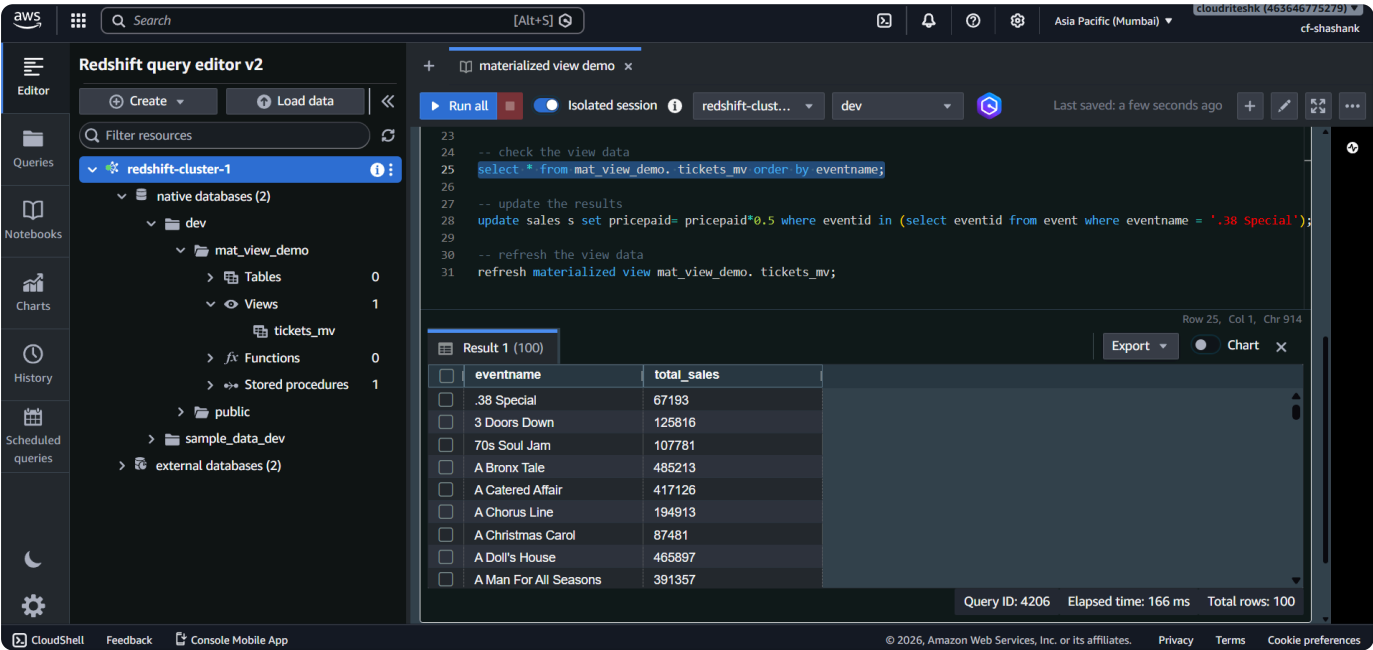
REFRESH MATERIALIZED [VIEW](#)

Purpose:

- Reload latest data from base tables.

Step 10 – Validate Refreshed Data

Queried the Materialized View again.



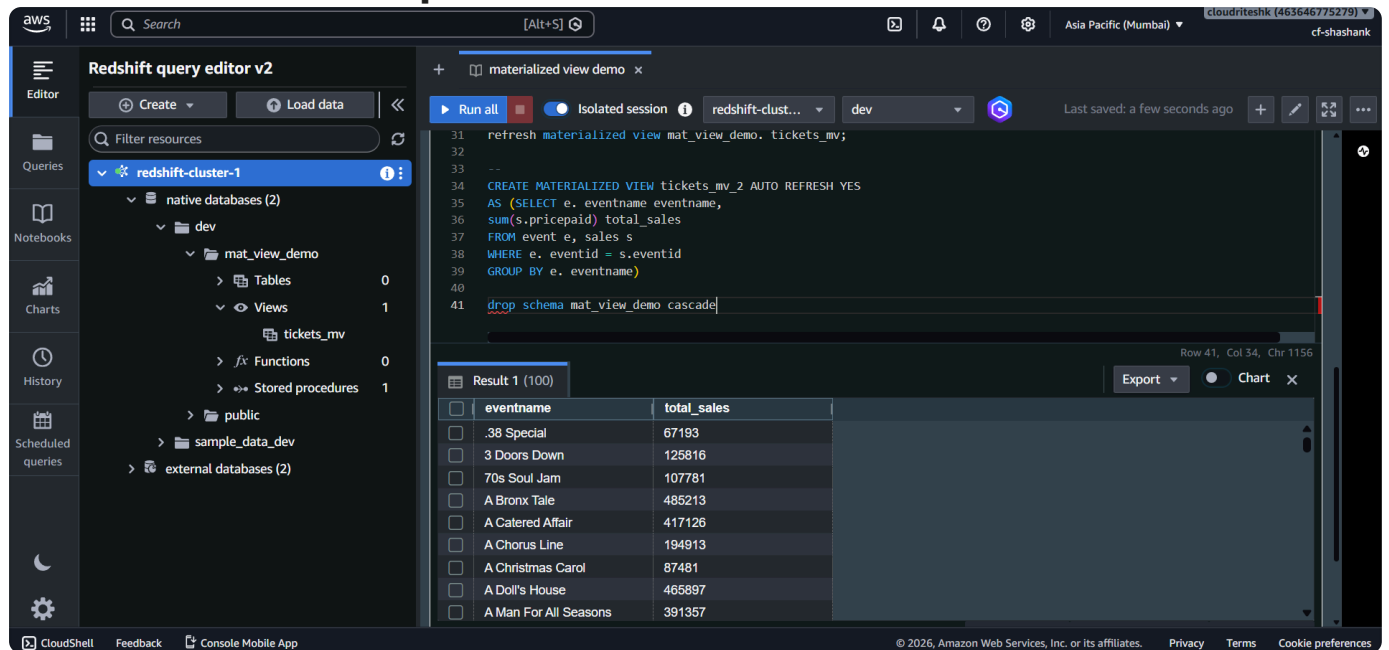
Result:

- Event: `.38 Special`

now showed: `0.5+ million sales`

This confirmed the refresh worked successfully.

Automatic Refresh Option



Materialized Views can also be configured for:

- Automatic refresh

With this:

- Redshift refreshes the view automatically after detecting table updates.
- Refresh is optimized to avoid affecting analytic workloads.

Key Learning

Materialized Views help:

- Cache expensive query results
- Improve analytics performance
- Reduce repeated query execution time

Best suited for:

- Reporting queries
- BI dashboards
- Repetitive aggregations and joins